DATA STRUCTURES

UNIT 4: TREE

EMPLOYEE ID MANAGEMENT USING BST

Employee ID and Name Management with Duplicate Validation

MODIFIED DOCUMENTATION

Subject	Data Structures
Unit	Unit 4 – Tree
Project	Employee ID Management Using BST
Version	Modified
Programming Language	C++

Prepared for Second-Year Engineering – Data Structures Project Documentation

1. Introduction

A Binary Search Tree (BST) is a non-linear data structure that stores keys so that smaller keys are placed in the left subtree and larger keys in the right subtree. This modified employee management project stores both employee ID and employee name, checks duplicate IDs, searches for an employee, displays records in ascending ID order, and counts the comparisons made during search.

2. Problem Statement

Enhance the original BST-based employee management system to store employee ID and employee name, reject duplicate employee IDs, display complete employee records in ascending ID order, and report the number of comparisons required to search for an employee.

3. Objectives

- To understand the structure and properties of a Binary Search Tree.
- To insert employee records according to BST ordering rules.
- To search for a required employee ID.
- To display employee IDs in ascending order using inorder traversal.
- To store employee name along with each employee ID.
- To prevent duplicate employee IDs and count search comparisons.

4. Data Structure Used

Binary Search Tree (BST): Each node has a key and links to its left and right child. For each node, values in the left subtree are smaller and values in the right subtree are larger. Inorder traversal (Left, Root, Right) displays keys in ascending order.

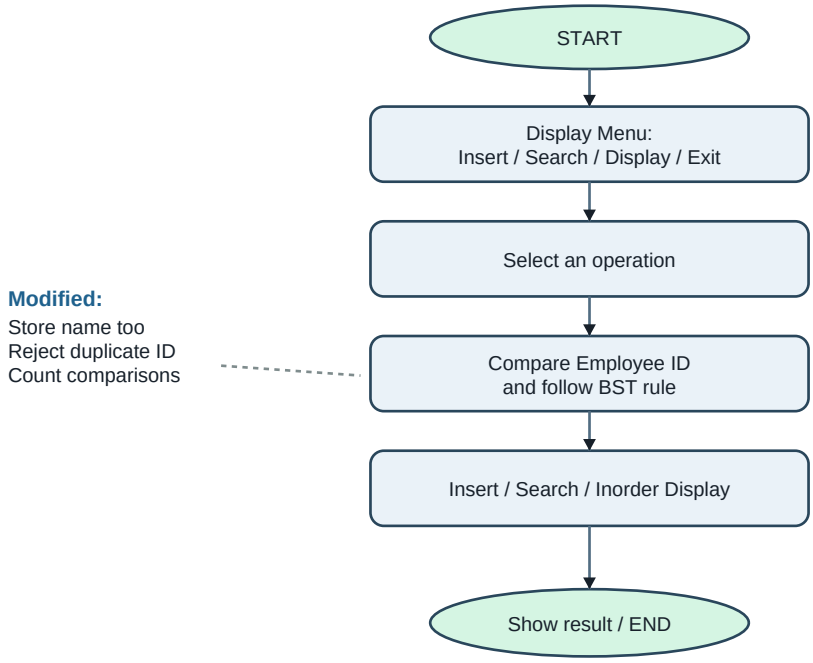
Node field	Purpose
Employee ID	Unique key used to arrange and search records.
Employee Name	Stores employee name.
Left pointer	Points to the left child containing smaller IDs.
Right pointer	Points to the right child containing larger IDs.

5. Algorithm

1. Start the program and initialize the BST root as NULL.
2. Display the menu: Insert, Search, Display Inorder, and Exit.
3. For insertion, read the employee ID and name.
4. If the tree is empty, create the root node; otherwise compare the ID with the current node.
5. If the ID is smaller, move to the left child; if larger, move to the right child.
6. If the ID already exists, reject the duplicate; otherwise insert the new record.
7. For search, compare the required ID with each visited node and move left or right accordingly.
8. For display, perform inorder traversal: Left subtree, Root, Right subtree.
9. Show the result and return to the menu until Exit is selected.
10. Stop the program.

6. Flowchart

The flowchart summarizes the main operations of the BST-based employee management system.



7. Working of the Project

The user enters an employee ID and name. The program compares the ID with existing BST nodes and places the new record in the left or right subtree according to its value. If an ID already exists, the program rejects it to avoid duplicate records. During searching, each visited node is compared with the target ID and the comparison count is updated. Inorder traversal displays all employee records in ascending order of ID.

8. Programming Language and Concepts

- Programming language: C++.
- Structures and pointers to create BST nodes.
- Dynamic memory allocation for new nodes.
- Recursion or iterative traversal for insertion, search, and inorder display.
- Conditional statements and loops to implement menu-driven operations.
- Duplicate-key validation and search-comparison counting.

9. Advantages

- Efficient searching and insertion when the BST is reasonably balanced.
- Inorder traversal displays employee IDs in sorted order.
- Dynamic memory allocation allows the tree to grow as records are added.
- Stores useful employee details, not only IDs.
- Duplicate IDs are detected and search effort can be measured.

10. Applications

- Employee record lookup and basic staff directories.
- Student ID or roll-number management.
- Understanding hierarchical data structures and ordered searching.
- Demonstrating duplicate validation and search-comparison analysis.

11. Sample Input and Output

Operation	Sample result
Insert	101 – Amit; 105 – Neha; 98 – Rahul
Search	Search ID: 105 → Record found: 105 – Neha
Comparisons	Search comparisons: 2 (for the sample tree)
Inorder display	98 – Rahul, 101 – Amit, 105 – Neha
Duplicate insert	101 → Duplicate employee ID; insertion rejected

12. Conclusion

The modified project extends BST employee management by storing names, validating unique IDs, and counting search comparisons. It demonstrates a more practical and informative use of a Binary Search Tree.

Time complexity: Average insertion/search is O(log n) in a reasonably balanced BST; worst case is O(n) for a skewed tree.